

Bit Packing V1.0

Projeto: ResQSense

Data: 21/03/2026

Autor: Luís Costa

Índice

1.	Objetivo	1
2.	Byte-Map (15 Bytes Fixos)	2
3.	Mapeamento Lógico (Packing/Unpacking)	2
4.	Fluxo de transmissão	3
5.	Experimentação	4
6.	Comentários finais	12

1. Objetivo

Estruturar e simular a comunicação entre as ESPS, o que futuramente vai funcionar por LoRa, com valores gerados aleatoriamente para os sinais vitais e localização e fixos para o id e network.

2. Byte-Map (15 Bytes Fixos)

Bytes	Tamanho	Descrição do Campo
0	1 Byte	Net ID + Vest ID
1	1 Byte	Flags de Estado
2 a 5	4 Byte	Latitude
6 a 9	4 Byte	Longitude
10 a 11	2 Byte	Altitude
12	1 Byte	BPMs
13 a 14	2 Byte	SpO2 + Temperatura

3. Mapeamento Lógico (Packing/Unpacking)

Cabeçalho e Identificação (Byte 0)

Junta-se dois identificadores num único byte usando operações de shift (<<):

- Net ID (3 bits): Limite de 0 a 7 redes (Máscara 0x07). Ocupa os 3 bits mais significativos.
- Vest ID (5 bits): Limite de 0 a 31 coletes (Máscara 0x1F). Ocupa os 5 bits menos significativos.

Máscara de Flags (Byte 1)

Oito booleanos (true/false) são empacotados num único byte.

- Bit 7 (MSB): Fixo a 1 (só para saber se é o pacote certo neste momento)
- Bits 6 a 0: Representam sequencialmente: SOS, Man Down, Vital Alert, Battery Low, Hardware Error, Pest Alert, e GPS Fix.

Geolocalização (Bytes 2 a 11)

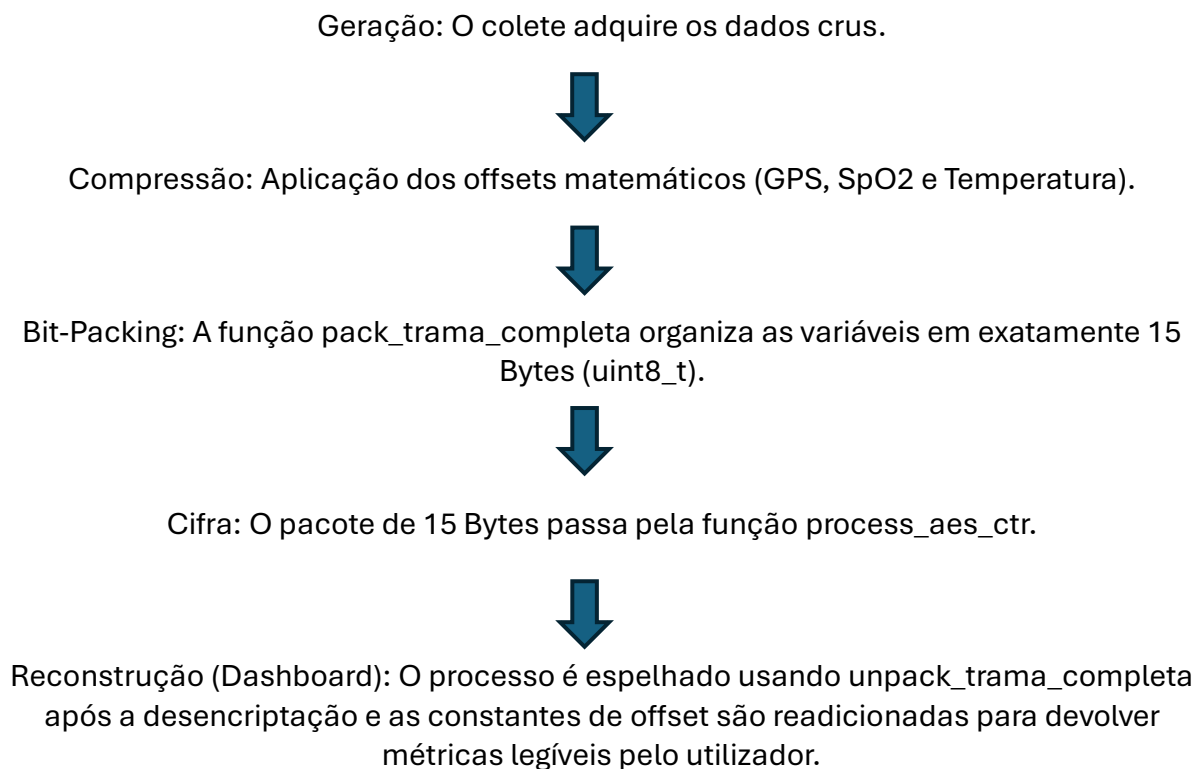
- **Latitude e Longitude** (4 Bytes cada): O valor em graus decimais (ex: 40.644000) é multiplicado por 10^7 e convertido num número inteiro com sinal (int32_t). Isto garante precisão ao centímetro e é dividido em 4 bytes consecutivos usando deslocamento de bits (>> 24, >> 16, >> 8, & 0xFF).
- **Altitude** (2 Bytes): Valor em metros guardado num (int16_t) direto.

Sinais Vitais Otimizados (Bytes 13 e 14)

Juntamos o nível de Oxigénio (SpO2) e a Temperatura do corpo em apenas 16 bits (2 bytes).

- **SpO2** (5 bits alocados): Um valor normal varia entre 70% e 100%. Subtraímos um offset de 69 ao valor original.
 - Exemplo: 98% de SpO2 torna-se $98 - 69 = 29$. O número 29 cabe perfeitamente em 5 bits (máximo 31).
 - Exceção: O valor 0 foi reservado para indicar um erro de leitura ou hipóxia grave ($< 70\%$).
- **Temperatura** (11 bits alocados): Em vez de enviar um float, aplicamos um offset focado no corpo humano e removemos as casas decimais.
 - Subtraímos 25.0°C ao valor lido e multiplicamos por 100.
 - Exemplo: 36.5°C torna-se $(36.5 - 25.0) * 100 = 1150$. 1150 cabe nos 11 bits restantes.
 - No dashboard, o processo é revertido: $(1150 / 100.0) + 25.0 = 36.5$.

4. Fluxo de transmissão



5. Experimentação

Testes dos códigos e o algoritmo do colete e do dashboard:

Usei duas ESPs, uma é a do projeto, a [ESP32S3 N8R8](#), e com uma que tinha em casa, uma [ESP](#) das mais básicas com 30 pins.

Código do colete:

```
#include <Arduino.h>
#include "mbedtls/aes.h"
#include <string.h>

/
// AES-128 CTR
const unsigned char AES_KEY[16] = {
    0x2B, 0x7E, 0x15, 0x16, 0x28, 0xAE, 0xD2, 0xA6,
    0xAB, 0xF7, 0x15, 0x88, 0x09, 0xCF, 0x4F, 0x3C
};

// Fixo para testes com fios.
// Num cenário real de rádio, isto deve ser sincronizado dinamicamente.
const unsigned char FIXED_IV[16] = {
    0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07,
    0x08, 0x09, 0x0A, 0x0B, 0x0C, 0x0D, 0x0E, 0x0F
};

// ESTRUTURA DE DADOS
struct SimulatedData {
    uint8_t bpm;
    uint8_t spo2;
    float temp;
    float lat;
    float lon;
    bool is_critical;
};
```

```

};

// Estrutura Lógica Intermédia (Antes do empacotamento)
struct ResQSenseData {
    uint8_t net_id;
    uint8_t vest_id;
    bool flag_sos, flag_man_down, flag_vital, flag_bat, flag_hw, flag_pest, flag_fix;
    int32_t lat_abs, lon_abs;
    int16_t altitude;
    uint8_t bpm, spo2;
    uint16_t temp_raw;
};

float randomFloat(float minVal, float maxVal) {
    return minVal + ((float)random(0, 10001) / 10000.0) * (maxVal - minVal);
}

void generateFakeData(SimulatedData* data) {
    bool happens_critical = (random(0, 100) < 10); // 10% de probabilidade de alerta
    data->is_critical = happens_critical;

    if (happens_critical) {
        int problem_type = random(0, 3);
        if (problem_type == 0) { // hr_high
            data->bpm = random(110, 161); data->spo2 = random(95, 101); data->temp = randomFloat(36.5, 37.5);
        } else if (problem_type == 1) { // spo2_low
            data->bpm = random(60, 101); data->spo2 = random(80, 95); data->temp = randomFloat(36.5, 37.5);
        } else { // temp_high
            data->bpm = random(90, 121); data->spo2 = random(95, 101); data->temp = randomFloat(41.0, 43.1);
        }
    } else {
        data->bpm = random(60, 91); data->spo2 = random(97, 101); data->temp = randomFloat(36.2, 37.2);
    }

    data->lat = 40.644 + randomFloat(-0.0001, 0.0001);
    data->lon = -8.645 + randomFloat(-0.0001, 0.0001);
}

// FUNÇÕES DE PROCESSAMENTO (BIT-PACKING E AES)

```

```

void pack_trama_completa(const struct ResQSenseData* data, uint8_t* buffer) {
    // Byte 0: Net ID (3 bits) e Vest ID (5 bits) [cite: 181, 182]
    buffer[0] = ((data->net_id & 0x07) << 5) | (data->vest_id & 0x1F);

    // Byte 1: Flags -> T(1) S M V B H P F [cite: 185, 186, 187, 188, 189, 192, 194, 195]
    buffer[1] = (1 << 7) | ((data->flag_sos & 0x01) << 6) | ((data->flag_man_down & 0x01) << 5) |
        ((data->flag_vital & 0x01) << 4) | ((data->flag_bat & 0x01) << 3) |
        ((data->flag_hw & 0x01) << 2) | ((data->flag_pest & 0x01) << 1) | (data->flag_fix & 0x01);

    // Bytes 2-5: Latitude Absoluta [cite: 201]
    buffer[2] = (data->lat_abs >> 24) & 0xFF; buffer[3] = (data->lat_abs >> 16) & 0xFF;
    buffer[4] = (data->lat_abs >> 8) & 0xFF; buffer[5] = data->lat_abs & 0xFF;

    // Bytes 6-9: Longitude Absoluta [cite: 201]
    buffer[6] = (data->lon_abs >> 24) & 0xFF; buffer[7] = (data->lon_abs >> 16) & 0xFF;
    buffer[8] = (data->lon_abs >> 8) & 0xFF; buffer[9] = data->lon_abs & 0xFF;

    // Bytes 10-11: Altitude [cite: 202]
    buffer[10] = (data->altitude >> 8) & 0xFF;
    buffer[11] = data->altitude & 0xFF;

    // Byte 12: BPM [cite: 203]
    buffer[12] = data->bpm;

    // Bytes 13-14: SpO2 (5 bits) + Temp (11 bits) [cite: 204, 207]
    uint16_t spo2_temp = ((data->spo2 & 0x1F) << 11) | (data->temp_raw & 0x07FF);
    buffer[13] = (spo2_temp >> 8) & 0xFF;
    buffer[14] = spo2_temp & 0xFF;
}

void process_aes_ctr(const uint8_t* input, uint8_t* output, size_t length) {
    mbedtls_aes_context aes;
    mbedtls_aes_init(&aes);
    mbedtls_aes_setkey_enc(&aes, AES_KEY, 128);

    size_t nc_off = 0;
    unsigned char stream_block[16] = {0};
    unsigned char nonce_counter[16];
    memcpy(nonce_counter, FIXED_IV, 16);

```

```

    mbedtls_aes_crypt_ctr(&aes, length, &nc_off, nonce_counter, stream_block, input, output);
    mbedtls_aes_free(&aes);
}

// VARIÁVEIS GLOBAIS
SimulatedData dados_crus;
struct ResQSenseData meus_dados;
uint8_t buffer_payload[15];
uint8_t buffer_cifrado[15];

void setup() {
    Serial.begin(115200);
    Serial2.begin(115200, SERIAL_8N1, 16, 17);
    randomSeed(analogRead(34));

    // Exemplo de configurações do colete
    meus_dados.net_id = 1;
    meus_dados.vest_id = 3;
    meus_dados.flag_sos = false;
    meus_dados.flag_man_down = false;
    meus_dados.flag_bat = false;
    meus_dados.flag_hw = false;
    meus_dados.flag_pest = false;
    meus_dados.flag_fix = true; // Assumimos que tem sinal GPS
    meus_dados.altitude = 15;

    Serial.println("Transmissor Iniciado! A enviar dados a cada 5 segundos...");
}

void loop() {
    // 1. Gera os dados
    generateFakeData(&dados_crus);

    // 2. Mapeia para a estrutura de empacotamento aplicando as janelas clínicas
    meus_dados.bpm = dados_crus.bpm;

    // SpO2: Mapeia de 70-100% para 1-31. Valor 0 é reservado para erro/hipóxia[cite: 205, 206].
    if (dados_crus.spo2 < 70) meus_dados.spo2 = 0;
    else meus_dados.spo2 = dados_crus.spo2 - 69;
}

```

```

// Temperatura: Offset de 25.00°C, multiplicado por 100 [cite: 207]
meus_dados.temp_raw = (uint16_t)((dados_crus.temp - 25.0) * 100.0);

// GPS: Graus decimais multiplicados por 10^7 [cite: 201]
meus_dados.lat_abs = (int32_t)(dados_crus.lat * 10000000.0);
meus_dados.lon_abs = (int32_t)(dados_crus.lon * 10000000.0);

// Flag Vital
meus_dados.flag_vital = dados_crus.is_critical;

// 3. Bit-Packing (Gera exatamente 15 Bytes)
pack_trama_completa(&meus_dados, buffer_payload);

// 4. Encriptar com AES-128 CTR
process_aes_ctr(buffer_payload, buffer_cifrado, 15);

// 5. Transmitir (Simulando o meio físico)
Serial2.write(buffer_cifrado, 15);

// Imprimir log de atividade no PC
Serial.println("=====");
Serial.println("PACOTE ENVIADO PELA SERIAL2");
Serial.print("Alerta Vital? "); Serial.println(dados_crus.is_critical ? "SIM" : "NÃO");
Serial.print("BPM Original: "); Serial.println(dados_crus.bpm);
Serial.print("SpO2 Original: "); Serial.println(dados_crus.spo2);
Serial.print("Temp Original: "); Serial.println(dados_crus.temp);
Serial.println("=====\\n");

delay(10000);
}

```

Codigo do Dashboard:

```

#include <Arduino.h>
#include "mbedtls/aes.h"
#include <string.h>

```



```

// AES-128 CTR
const unsigned char AES_KEY[16] = {
    0x2B, 0x7E, 0x15, 0x16, 0x28, 0xAE, 0xD2, 0xA6,
    0xAB, 0xF7, 0x15, 0x88, 0x09, 0xCF, 0x4F, 0x3C
};

const unsigned char FIXED_IV[16] = {
    0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07,
    0x08, 0x09, 0x0A, 0x0B, 0x0C, 0x0D, 0x0E, 0x0F
};

// Estrutura de dados
struct ResQSenseData {
    uint8_t net_id;
    uint8_t vest_id;
    bool flag_sos, flag_man_down, flag_vital, flag_bat, flag_hw, flag_pest, flag_fix;
    int32_t lat_abs, lon_abs;
    int16_t altitude;
    uint8_t bpm, spo2;
    uint16_t temp_raw;
};

// FUNÇÃO para encriptar e desencriptar o AES-128 CTR
void process_aes_ctr(const uint8_t* input, uint8_t* output, size_t length) {
    mbedtls_aes_context aes;
    mbedtls_aes_init(&aes);
    mbedtls_aes_setkey_enc(&aes, AES_KEY, 128);

    size_t nc_off = 0;
    unsigned char stream_block[16] = {0};
    unsigned char nonce_counter[16];
    memcpy(nonce_counter, FIXED_IV, 16);

    mbedtls_aes_crypt_ctr(&aes, length, &nc_off, nonce_counter, stream_block, input, output);
    mbedtls_aes_free(&aes);
}

// FUNÇÃO de Bit-UNPACKING
void unpack_trama_completa(const uint8_t* buffer, struct ResQSenseData* data) {

```

```

// Byte 0: Net ID (3 bits) e Vest ID (5 bits)
data->net_id = (buffer[0] >> 5) & 0x07;
data->vest_id = buffer[0] & 0x1F;

// Byte 1: Flags
data->flag_sos    = (buffer[1] >> 6) & 0x01;
data->flag_man_down = (buffer[1] >> 5) & 0x01;
data->flag_vital   = (buffer[1] >> 4) & 0x01;
data->flag_bat     = (buffer[1] >> 3) & 0x01;
data->flag_hw      = (buffer[1] >> 2) & 0x01;
data->flag_pest    = (buffer[1] >> 1) & 0x01;
data->flag_fix     = buffer[1] & 0x01;

// Bytes 2-5: Latitude (Reconstruir os 32 bits)
data->lat_abs = ((int32_t)buffer[2] << 24) | ((int32_t)buffer[3] << 16) |
               ((int32_t)buffer[4] << 8) | buffer[5];

// Bytes 6-9: Longitude (Reconstruir os 32 bits)
data->lon_abs = ((int32_t)buffer[6] << 24) | ((int32_t)buffer[7] << 16) |
               ((int32_t)buffer[8] << 8) | buffer[9];

// Bytes 10-11: Altitude (Reconstruir os 16 bits)
data->altitude = ((int16_t)buffer[10] << 8) | buffer[11];

// Byte 12: BPM
data->bpm = buffer[12];

// Bytes 13-14: SpO2 (5 bits) e Temperatura (11 bits)
uint16_t spo2_temp = ((uint16_t)buffer[13] << 8) | buffer[14];
data->spo2 = (spo2_temp >> 11) & 0x1F;
data->temp_raw = spo2_temp & 0x07FF;
}

// VARIÁVEIS GLOBAIS
uint8_t rx_buffer_cifrado[15];
uint8_t rx_buffer_limpo[15];
struct ResQSenseData dados_recebidos;

void setup() {
    Serial.begin(115200);

```

```

Serial2.begin(115200, SERIAL_8N1, 16, 17);

Serial.println("\n--- Dashboard Tablet Iniciado ---");
Serial.println("A aguardar transmissões da equipa...");
}

void loop() {
    // Verifica se recebemos a trama completa de 15 bytes na porta série
    if (Serial2.available() >= 15) {

        // Lê os exatos 15 bytes
        Serial2.readBytes(rx_buffer_cifrado, 15);

        // 1. Descriptar a Payload
        process_aes_ctr(rx_buffer_cifrado, rx_buffer_limpo, 15);

        // Verifica se é uma Trama Completa (O Bit 7 do Byte 1 tem de ser 1)
        if ((rx_buffer_limpo[1] & 0x80) != 0x80) {
            Serial.println("Erro: Trama recebida não é do tipo Completa (Telemetria).");
            return;
        }

        // 2. Extrair os bits para a Struct
        unpack_trama_completa(rx_buffer_limpo, &dados_recebidos);

        // 3. Converter de volta para as escalas humanas (Conforme a Especificação)
        int real_spo2 = (dados_recebidos.spo2 == 0) ? 0 : dados_recebidos.spo2 + 69;
        float real_temp = (dados_recebidos.temp_raw / 100.0) + 25.0;
        float real_lat = dados_recebidos.lat_abs / 10000000.0;
        float real_lon = dados_recebidos.lon_abs / 10000000.0;

        // 4. Imprimir no Dashboard (Monitor Serial)
        Serial.println("=====");
        Serial.print("PACOTE RECEBIDO | Net ID: "); Serial.print(dados_recebidos.net_id);
        Serial.print(" | Colete ID: "); Serial.println(dados_recebidos.vest_id);

        if (dados_recebidos.flag_vital) {
            Serial.println("⚠ ALERTA VITAL ACIONADO ⚠");
        }
    }
}

```

```

Serial.print("BPM: "); Serial.println(dados_recebidos.bpm);

Serial.print("SpO2: ");
if (real_spo2 == 0) Serial.println("ERRO/HIPÓXIA (<70%)");
else { Serial.print(real_spo2); Serial.println("%"); }

Serial.print("Temp: "); Serial.print(real_temp, 2); Serial.println(" °C");

Serial.print("GPS: "); Serial.print(real_lat, 6); Serial.print(", "); Serial.println(real_lon, 6);
Serial.println("=====\n");
}
}

```

Em ambas as ESPs configurei o RX e o TX nos pinos 16 e 17, respetivamente, ou seja, liguei as ESPs via UART, partilhando o GND e ligando o TX do colete ligado ao RX do dashboard, ou seja, GND(colete) → GND(dashboard) e GPIO17(colete) → GPIO16(dashboard).

Durante os uploads dos códigos notei que falhou algumas vezes quando as espes estavam ligadas entre si antes de dar upload, para resolver, o que fiz foi desligar as ligações, dar upload do colete e desligar, depois upload do código do dashboard e por fim fazemos as ligações entre eles.

Nem sempre aconteceu, mas só assim é que consegui resolver o problema do uploading nas primeiras tentativas.

Usei a UART por ser um protocolo assíncrono, o que significa que envia bytes “em série”, sem depender de um relógio partilhado (como acontece no I2C ou SPI). Uma transmissão LoRa funciona muito semelhantemente a este mesmo princípio: “lançar” os bytes para o canal e esperar que o outro lado os “apanhe”.

6. Comentários finais

- O objetivo agora é substituir a ligação física via UART pelos módulos LoRa.

- A usar UART não há grandes interferências, mas no ar o cenário é diferente.